DBMS Laboratory — Project Report

Reimbursement Management System

A database application for employee expense claims, approvals and payments

Submitted by

MIS number	Name
612415001	Abdurrahman Shaikh
612415057	Dhruv Agarwal
612415044	Mrudula Dagale
612415093	Makarand S Kulkarni

Database	MySQL 8
Back end	Node.js, Express (REST API, plain SQL)
Front end	React, Tailwind CSS
Source code (File 1)	https://github.com/Abdurrahman-shaikh02/reimbursement-management

29 October 2026

Contents

1 Problem statement 3

2 Objectives 3

3 Functional requirements 3

3.1 Business rules 4

4 ER diagram 4

4.1 Entity sets 4

4.2 Relationship sets 5

4.3 Main design choices 5

4.4 Changes from the earlier ER diagram 7

5 Relational schemas obtained from the ER diagram 7

5.1 Schema diagram 8

5.2 How the reduction was done 9

5.3 Keys and foreign keys 9

5.4 Integrity constraints 9

6 Functional dependencies on each table 10

6.1 Dependencies that do not hold 11

7 Normalization up to 3NF 11

7.1 Definitions used 11

7.2 Starting point: one flat relation 11

7.3 First normal form 12

7.4 Second normal form 12

7.5 Third normal form 12

7.6 Normal form of each final table 13

7.7 The decomposition is lossless and dependency-preserving 13

8 Implementation 13

9 Conclusion 15

1 Problem statement

Employees regularly spend their own money on company work: travelling to a client, staying in a hotel during a business trip, or paying a medical bill that the company policy covers. In many organisations these claims still move as paper forms and spreadsheets. Bills get lost, nobody can tell what state a claim is in, a manager can approve an amount beyond their authority (or even their own claim), and the finance team cannot easily answer questions such as "how much did each department spend on travel this quarter?".

The aim of this project is a database application in which an employee files a typed claim (travel, medical or business) made up of individual bills with receipts. A manager other than the claimant approves or rejects it within their approval limit, giving a reason for a rejection. An approved claim is then marked paid into the claimant's registered bank account. Every claim and every decision is stored consistently and kept as a financial record.

The project builds on the ER diagram prepared for the earlier ER-diagram assignment (Reimbursement Management).

2 Objectives

1. Store employees, departments, reporting lines and roles (applicant, manager) without redundancy.
2. Record each claim with its type-specific details and its individual bills, each with an optional receipt.
3. Implement the approval workflow: SUBMITTED → APPROVED or REJECTED, and APPROVED → PAID, with no self-approval, approval only within the manager's limit, and a reason for every rejection.
4. Enforce data integrity with primary keys, candidate keys, foreign keys, NOT NULL and CHECK constraints in the database.
5. Design every table in at least third normal form (3NF), so that there are no update, insertion or deletion anomalies.
6. Provide reports: totals by department, claim type, expense category and month, and claims by status.
7. Provide a simple web front end for applicants and managers on top of the database.

3 Functional requirements

The system has two roles. An employee may have one role, both, or neither: a manager may also file their own claims.

- **Applicant:** an employee with a registered bank account, who files claims.
- **Manager:** an employee with an approval limit, who decides claims, marks them paid, maintains departments and employees, and sees the reports.

#	Requirement	Tables used
FR1	An employee logs in with email and password.	employee
FR2	The system knows each user's roles: applicant, manager, both or neither.	applicant, manager
FR3	Managers add and edit departments and employees, with their phone numbers, supervisor and roles.	department, employee, employee_ phone, applicant, manager

#	Requirement	Tables used
FR4	An applicant submits a claim of exactly one type (travel, medical or business) with its type-specific details and one or more expense items, each with a category, description, amount, date and optional receipt.	reimbursement, travel_ / medical_ / business_reimbursement, expense_item
FR5	An applicant sees their claims with total, status, decision date and remarks.	reimbursement, expense_item (view reimbursement_summary)
FR6	A manager sees the submitted claims, other than their own, whose total is within their approval limit.	reimbursement, manager, expense_item
FR7	A manager approves or rejects a claim; a rejection requires remarks; the decision date is recorded.	reimbursement
FR8	A manager marks an approved claim as paid (not their own claim).	reimbursement, applicant
FR9	Reports: totals by department, claim type, category and month; claims by status.	all tables, through joins and GROUP BY

3.1 Business rules

- A claim belongs to exactly one type and has at least one expense item.
- No expense may be dated after the claim date.
- A claim moves only SUBMITTED → APPROVED → PAID or SUBMITTED → REJECTED. A decision, once made, is not edited, and saved expense items are not changed.
- A manager cannot decide or pay their own claim, and can approve a claim only if its total is within their approval limit.
- An employee cannot supervise themselves, directly or through a chain of subordinates.

4 ER diagram

Figure 1 on the next page is the ER diagram, drawn in the notation of Silberschatz, Korth and Sudarshan, *Database System Concepts* (7th edition, Chapter 6). Attributes are listed inside the entity rectangles; primary keys are underlined; the key at the bottom of the figure explains every symbol.

4.1 Entity sets

Entity set	Attributes	Notes
DEPARTMENT	dept_id, dept_name, location	Strong entity set
EMPLOYEE	employee_id, name, email, password_hash, designation, date_of_joining, { phone }	Strong entity set; phone is multivalued
APPLICANT	bank_account_no	Subclass of EMPLOYEE
MANAGER	approval_limit	Subclass of EMPLOYEE
REIMBURSEMENT	reimbursement_id, claim_type, claim_date, status, total_amount ( )	Strong entity set; total_amount is derived
TRAVEL_REIMBURSEMENT	travel_from, travel_to, distance_km	Subclass of REIMBURSEMENT
MEDICAL_ REIMBURSEMENT	hospital_name, treatment_type	Subclass of REIMBURSEMENT

Entity set	Attributes	Notes
BUSINESS_ REIMBURSEMENT	purpose, client_name	Subclass of REIMBURSEMENT
EXPENSE_ITEM	item_no, category, description, amount, expense_date, receipt_path	Weak entity set; item_no is the discriminator

4.2 Relationship sets

Relationship	Between	Cardinality	Participation	Attributes
WORKS_IN	EMPLOYEE – DEPARTMENT	many-to-one	EMPLOYEE total, DEPARTMENT partial	—
SUPERVISES (recursive)	EMPLOYEE (supervisor) – EMPLOYEE (subordinate)	one-to-many	both partial	—
SUBMITS	APPLICANT – REIMBURSEMENT	one-to-many	REIMBURSEMENT total, APPLICANT partial	—
APPROVES	MANAGER – CLAIM_PROCESS (aggregation)	one-to-many	both partial	approval_date, remarks
CONTAINS (identifying)	REIMBURSEMENT – EXPENSE_ITEM	one-to-many	both total	—

4.3 Main design choices

- **Specialization of EMPLOYEE** into APPLICANT and MANAGER is **overlapping** (a manager may also file claims) and **partial** (some employees have neither role).
- **Specialization of REIMBURSEMENT** into TRAVEL_, MEDICAL_ and BUSINESS_REIMBURSEMENT is **disjoint** (a claim has exactly one purpose) and **total** (every claim has a type). The attribute claim_type says which subclass a claim belongs to.
- **Aggregation CLAIM_PROCESS:** the SUBMITS relationship (applicant submits claim) is treated as one higher-level entity, and MANAGER APPROVES it. What a manager decides on is a submission, and a ternary relationship would store the applicant twice (§6.8.5).
- **EXPENSE_ITEM is a weak entity set:** an item has no meaning outside its claim, so it is identified by the claim plus item_no (1, 2, 3 … within each claim) through the identifying relationship CONTAINS (§6.5.3).
- **DEPARTMENT is an entity set**, not a text attribute of EMPLOYEE, because it has an attribute of its own (location) and is shared by many employees (§6.9.2).
- **total_amount is derived** from the amounts of the claim's items, so it is not stored (§6.3). **phone is multivalued**, so it becomes a separate table.
- **approval_date and remarks belong to APPROVES**, because they describe the manager's decision and exist only when a decision exists.

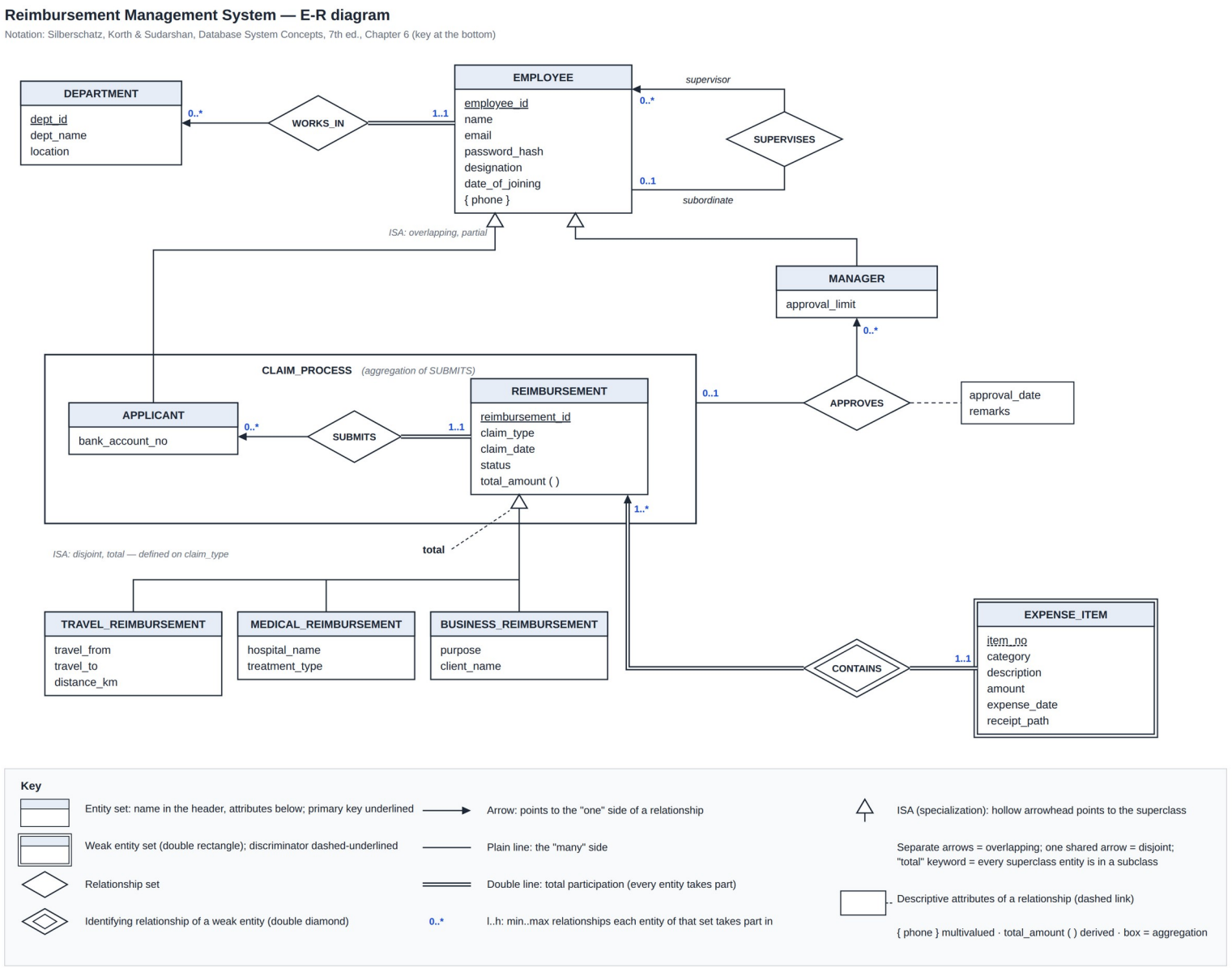


Figure 1. ER diagram of the Reimbursement Management System, with the key to the notation.

4.4 Changes from the earlier ER diagram

#	Change	Reason
1	approval_date and remarks added to APPROVES	A rejection needs a reason and the decision date must be recorded
2	Claim specialization marked disjoint and total; claim_type added	Each claim is exactly one type
3	Status values made explicit: SUBMITTED, APPROVED, REJECTED, PAID	Defines the workflow and allows a CHECK constraint
4	DEPARTMENT made an entity set (dept_id, dept_name, location)	Removes the transitive dependency employee → department → location
5	Receipt stored as a file path	Files stay on disk; the database stores only the path
6	category added to EXPENSE_ITEM	Grouping in reports
7	password_hash added to EMPLOYEE	Users must log in

5 Relational schemas obtained from the ER diagram

The ER diagram reduces to 10 relation schemas, using the reduction rules of the textbook (§6.7 and §6.8.6). Primary keys are underlined; foreign keys are listed in Section 5.3.

Relation schema	Obtained from
department (dept_id, dept_name, location)	Strong entity set DEPARTMENT
employee (employee_id, name, email, password_hash, designation, date_of_joining, dept_id, supervisor_id)	Strong entity set EMPLOYEE; many-to-one WORKS_IN and SUPERVISES folded in as dept_id and supervisor_id
employee_phone (employee_id, phone)	Multivalued attribute { phone }
applicant (employee_id, bank_account_no)	Subclass of EMPLOYEE: subclass attributes plus the superclass key
manager (employee_id, approval_limit)	Subclass of EMPLOYEE
reimbursement (reimbursement_id, applicant_id, claim_type, claim_date, status, approved_by, approval_date, remarks)	Strong entity set REIMBURSEMENT; SUBMITS folded in as applicant_id; APPROVES (on the aggregation) folded in as approved_by, approval_date, remarks
travel_reimbursement (reimbursement_id, travel_from, travel_to, distance_km)	Subclass of REIMBURSEMENT
medical_reimbursement (reimbursement_id, hospital_name, treatment_type)	Subclass of REIMBURSEMENT
business_reimbursement (reimbursement_id, purpose, client_name)	Subclass of REIMBURSEMENT
expense_item (reimbursement_id, item_no, category, description, amount, expense_date, receipt_path)	Weak entity set EXPENSE_ITEM: owner's key plus discriminator

View reimbursement_summary (reimbursement_id, …, total_amount, item_count) computes the derived attribute total_amount as SUM(expense_item.amount) for each claim, so the total is never stored twice.

5.1 Schema diagram

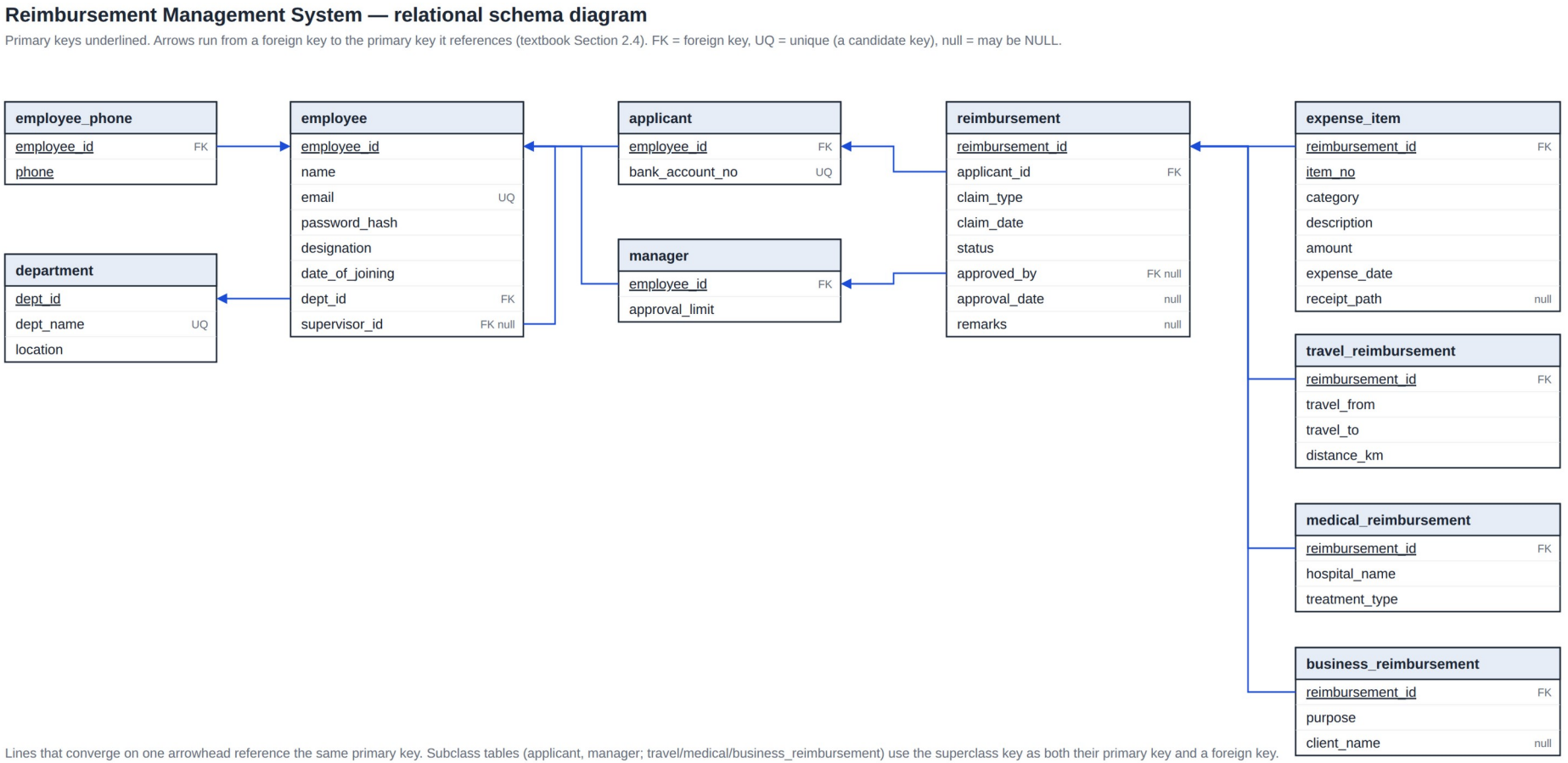


Figure 2. Relational schema diagram. Each arrow goes from a foreign key to the primary key it references.

5.2 How the reduction was done

- **Many-to-one relationships** (WORKS_IN, SUPERVISES, SUBMITS, APPROVES) get no table of their own. Each becomes a foreign key in the table on the "many" side (§6.7.6). Total participation makes the column NOT NULL (dept_id, applicant_id); partial participation leaves it nullable (supervisor_id, approved_by).
- **CONTAINS** needs no table: its schema (reimbursement_id, item_no) is already inside expense_item (§6.7.5). The **aggregation** CLAIM_PROCESS uses the table of its relationship SUBMITS, which is reimbursement (§6.8.6.2).
- **Both specializations** use one table for the superclass and one per subclass, holding the subclass attributes plus the superclass key as primary key and foreign key (§6.8.6.1). For EMPLOYEE this is required because the specialization is overlapping and partial. For REIMBURSEMENT it lets expense_item and approved_by refer to a single claim table.
- **The multivalued attribute** phone becomes employee_phone, whose key is the whole schema (§6.7.2).

5.3 Keys and foreign keys

Table	Primary key	Candidate keys (UNIQUE)	Foreign keys (ON DELETE)
department	dept_id	dept_name	—
employee	employee_id	email	dept_id → department (RESTRICT); supervisor_id → employee (RESTRICT)
employee_phone	(employee_id, phone)	—	employee_id → employee (CASCADE)
applicant	employee_id	bank_account_no	employee_id → employee (CASCADE)
manager	employee_id	—	employee_id → employee (CASCADE)
reimbursement	reimbursement_id	—	applicant_id → applicant (RESTRICT); approved_by → manager (RESTRICT)
travel_ / medical_ / business_reimbursement	reimbursement_id	—	reimbursement_id → reimbursement (CASCADE)
expense_item	(reimbursement_id, item_no)	—	reimbursement_id → reimbursement (CASCADE)

CASCADE is used only where the child row is part of the parent (phones, role rows, claim details, expense items). Everywhere else the delete is RESTRICTed: claims are financial records and must be kept.

5.4 Integrity constraints

Besides the keys above and NOT NULL on every mandatory column, the tables declare these CHECK constraints:

Table	CHECK constraint	Meaning
reimbursement	status IN ('SUBMITTED', 'APPROVED', 'REJECTED', 'PAID')	Only the four workflow states
reimbursement	claim_type IN ('TRAVEL', 'MEDICAL', 'BUSINESS')	Only the three claim types

Table	CHECK constraint	Meaning
reimbursement	(status = 'SUBMITTED' AND approved_by IS NULL AND approval_date IS NULL) OR (status <> 'SUBMITTED' AND approved_by IS NOT NULL AND approval_date IS NOT NULL)	A decision is recorded exactly when the claim has left SUBMITTED
reimbursement	status <> 'REJECTED' OR remarks is not empty	A rejection needs a reason
reimbursement	approved_by IS NULL OR approved_by <> applicant_id	No one decides their own claim
reimbursement	approval_date IS NULL OR approval_date >= claim_date	No decision before the claim is filed
expense_item	category IN ('FOOD', 'LODGING', 'FARE', 'FUEL', 'LOCAL_ TRANSPORT', 'MEDICAL', 'SUPPLIES', 'OTHER')	Fixed categories for reports
expense_item, manager, travel_ reimbursement	amount > 0, approval_limit > 0, distance_km > 0	No zero or negative values
employee_phone	7 to 15 characters, an optional + followed by digits	Phone number format
employee	email LIKE '%_@_%._%'	Email format (the login name)

Some business rules need the previous value of a row or data from another table: the allowed status changes, the approval limit, "expense date not after the claim date", the claim type matching its details table, and "no supervision cycles". MySQL does not allow subqueries in a CHECK constraint and has no assertions (§4.4.8), so the application checks these rules. The same validation code runs in the web form and again in the API, inside the transaction that writes the data. The API reads the rows the rule depends on with SELECT … FOR UPDATE, so two concurrent requests cannot both pass the check. A claim, its details row and its items are inserted in one transaction (§4.3), so a claim is never stored without them.

6 Functional dependencies on each table

For each table, the non-trivial functional dependencies that must hold (a canonical cover) and the candidate keys they give. Each table has at most one line per left-hand side.

Table	Functional dependencies	Candidate keys
department	dept_id → dept_name, location dept_name → dept_id	{dept_id}, {dept_name}
employee	employee_id → name, email, password_hash, designation, date_ of_joining, dept_id, supervisor_id email → employee_id	{employee_id}, {email}
employee_phone	only trivial FDs (the key is the whole schema)	{employee_id, phone}
applicant	employee_id → bank_account_no bank_account_no → employee_id	{employee_id}, {bank_ account_no}
manager	employee_id → approval_limit	{employee_id}
reimbursement	reimbursement_id → applicant_id, claim_type, claim_date, status, approved_by, approval_date, remarks	{reimbursement_id}
travel_reimbursement	reimbursement_id → travel_from, travel_to, distance_km	{reimbursement_id}
medical_reimbursement	reimbursement_id → hospital_name, treatment_type	{reimbursement_id}
business_reimbursement	reimbursement_id → purpose, client_name	{reimbursement_id}

Table	Functional dependencies	Candidate keys
expense_item	(reimbursement_id, item_no) → category, description, amount, expense_date, receipt_path	{reimbursement_id, item_no}

Every FD above is enforced by the database: when the left side is the primary key, by the PRIMARY KEY constraint; when it is another candidate key (dept_name, email, bank_account_no), by a UNIQUE constraint.

6.1 Dependencies that do not hold

The following might look like functional dependencies, but they do not hold for this organisation, so they are not asserted:

Not an FD	Reason
(travel_from, travel_to) → distance_km	distance_km is the distance of this trip as claimed (route taken, detours, round trip), not a property of the two places. If the company paid a fixed distance per city pair, this FD would hold and travel_reimbursement would have to be decomposed into route (travel_from, travel_to, distance_km).
designation → approval_limit	Limits are given to individual managers; two managers with the same designation can have different limits.
dept_id → supervisor_id	A department can have several supervisors (team leads).
hospital_name → treatment_type	The same hospital treats many conditions.
applicant_id → claim_type, approved_by → approval_date	An applicant files many claims; a manager decides many claims on different dates.

7 Normalization up to 3NF

Result: every table is in Boyce–Codd normal form (BCNF), and therefore also in 3NF. This section first normalizes a single "claim form" relation step by step (1NF → 2NF → 3NF) and shows that it gives the same tables as the ER reduction. It then checks each final table.

7.1 Definitions used

- **1NF:** every attribute domain is atomic (no lists or repeating groups) (§7.8).
- **2NF:** in 1NF, and no non-prime attribute depends on a proper subset of a candidate key.
- **3NF:** for every FD α → β, either it is trivial, or α is a superkey, or every attribute in β − α is part of some candidate key (§7.3.2).
- **BCNF:** for every non-trivial FD α → β, α is a superkey (§7.3.1). Every BCNF schema is also in 3NF.

7.2 Starting point: one flat relation

Suppose all claim data were kept in one relation, the way a paper form or a spreadsheet keeps it (the type-specific details are left out; they follow the same pattern):

```
claim_form (rid, claim_date, claim_type, status,
            applicant_id, name, email, bank_account_no, dept_id, dept_name, location, phones,
            approved_by, approval_limit, approval_date, remarks,
            item_no, category, description, amount, expense_date, receipt_path)
```

Here rid stands for reimbursement_id. This relation has all three anomalies (§7.1):

- **Update anomaly:** an employee's email is repeated on every item of every claim; changing it in one row leaves the other rows wrong.
- **Insertion anomaly:** a new department cannot be stored until somebody in it files a claim.
- **Deletion anomaly:** deleting the only claim of a department also deletes the department's location.

Functional dependencies F that hold on claim_form:

```
F1  rid             → claim_date, claim_type, status, applicant_id,
                      approved_by, approval_date, remarks
F2  applicant_id    → name, email, bank_account_no, dept_id
F3  email           → applicant_id
F4  bank_account_no → applicant_id
F5  dept_id         → dept_name, location
F6  dept_name       → dept_id
F7  approved_by     → approval_limit
F8  rid, item_no    → category, description, amount, expense_date, receipt_path
```

7.3 First normal form

phones holds a list of numbers, so claim_form is not in 1NF. We move the multivalued attribute into its own relation, **employee_phone (employee_id, phone)**. What remains has only atomic values. Its only candidate key is (rid, item_no), because the closure (rid, item_no)+ under F contains every attribute.

7.4 Second normal form

By F1, all the claim attributes depend on rid alone, which is a proper subset of the key (rid, item_no). This partial dependency violates 2NF. Splitting on rid gives:

- **expense_item (rid, item_no, category, description, amount, expense_date, receipt_path)**: F8, key (rid, item_no).
- **claim (rid, claim_date, claim_type, status, applicant_id, name, email, bank_account_no, dept_id, dept_name, location, approved_by, approval_limit, approval_date, remarks)**: key rid.

7.5 Third normal form

claim still has transitive dependencies: rid → applicant_id → name, email, bank_account_no, dept_id (F2); then dept_id → dept_name, location (F5); and rid → approved_by → approval_limit (F7). applicant_id, dept_id and approved_by are not superkeys of claim, and the attributes they determine are not part of any candidate key, so claim violates 3NF. Decomposing on each of these FDs gives:

- **employee (applicant_id, name, email, bank_account_no, dept_id)** from F2, and **department (dept_id, dept_name, location)** from F5.
- **manager (approved_by, approval_limit)** from F7.
- **reimbursement (rid, claim_date, claim_type, status, applicant_id, approved_by, approval_date, remarks)**: what remains of claim.

These are the tables obtained from the ER diagram in Section 5. The only difference is that the final design keeps bank_account_no in applicant rather than employee, because only applicants have a payout account. That comes from the specialization, not from normalization, and both placements are in BCNF.

The same result follows from the textbook's 3NF synthesis algorithm (§7.5.2). One schema per FD of the canonical cover gives F1, F2, F5, F7 and F8 as schemas. The schemas for F3, F4 and F6 are

contained in those of F2 and F5, so they are dropped. F8 contains the candidate key (rid, item_no), so no extra schema is needed.

7.6 Normal form of each final table

In every table of Section 6, the left side of every non-trivial FD is a candidate key, and so a superkey. That is the BCNF condition.

Table	1NF	2NF	3NF	BCNF	Reason
department	✓	✓	✓	✓	dept_id and dept_name are both candidate keys
employee	✓	✓	✓	✓	employee_id and email are both candidate keys; department details moved out
employee_phone	✓	✓	✓	✓	All-key relation: no non-trivial FD
applicant	✓	✓	✓	✓	employee_id and bank_account_no are both candidate keys
manager	✓	✓	✓	✓	Single FD from the key
reimbursement	✓	✓	✓	✓	Single-attribute key; applicant and approver details moved out
travel_reimbursement	✓	✓	✓	✓	Single FD from the key (see Section 6.1 on distance_km)
medical_reimbursement	✓	✓	✓	✓	Single FD from the key
business_reimbursement	✓	✓	✓	✓	Single FD from the key
expense_item	✓	✓	✓	✓	Every non-key attribute depends on the whole key (reimbursement_id, item_no)

7.7 The decomposition is lossless and dependency-preserving

- **Lossless join (§7.2.3):** at every split, the common attributes are a key of one of the two parts. For example, claim ∩ expense_item = {rid}, the key of claim, and claim ∩ employee = {applicant_id}, the key of employee. So joining the tables gives back exactly the original data.
- **Dependency-preserving (§7.4.4):** each of F1 to F8 has all its attributes inside one final table, so each can be checked within that table by its PRIMARY KEY or UNIQUE constraint, without a join.

8 Implementation

The database is implemented in MySQL 8 (version 8.0.16 or later, which enforces CHECK constraints). The SQL scripts in db/ create the tables, the reimbursement_summary view, an application database user and sample data, and test every constraint with invalid data. The back end is a Node.js + Express REST API that uses plain SQL through the mysql2 driver. The front end is a React web application. The complete source code and screenshots of all the forms are in File 1 and on GitHub:

https://github.com/Abdurrahman-shaikh02/reimbursement-management

A few of the forms:

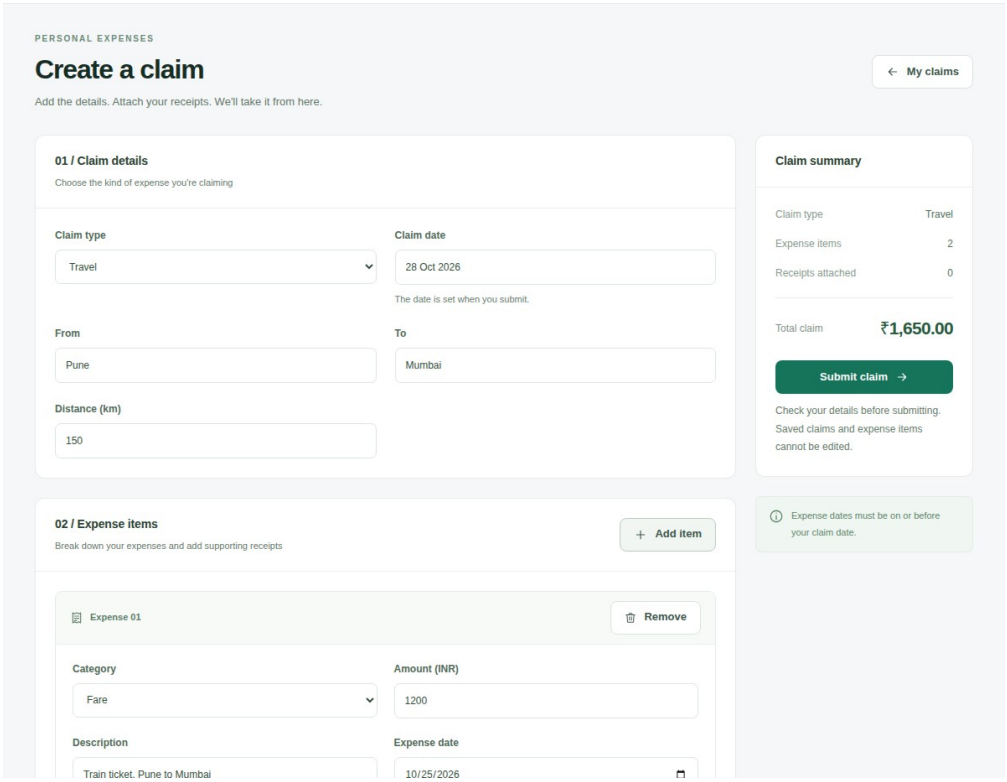


Figure 3. An applicant creates a travel claim with two expense items.

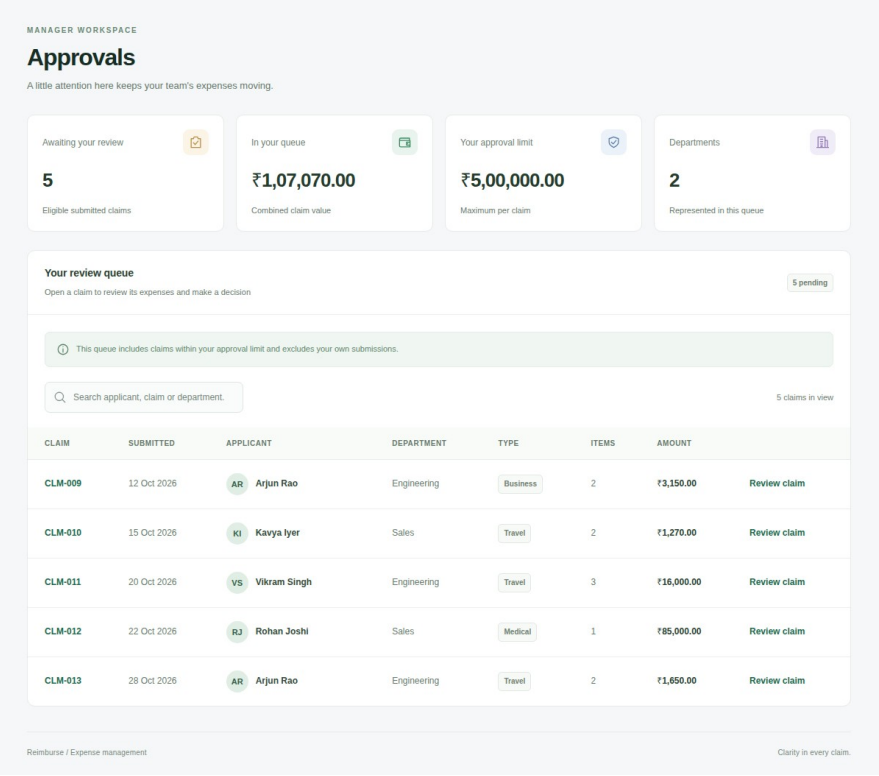


Figure 4. A manager's approval queue: submitted claims, not their own, within their approval limit.

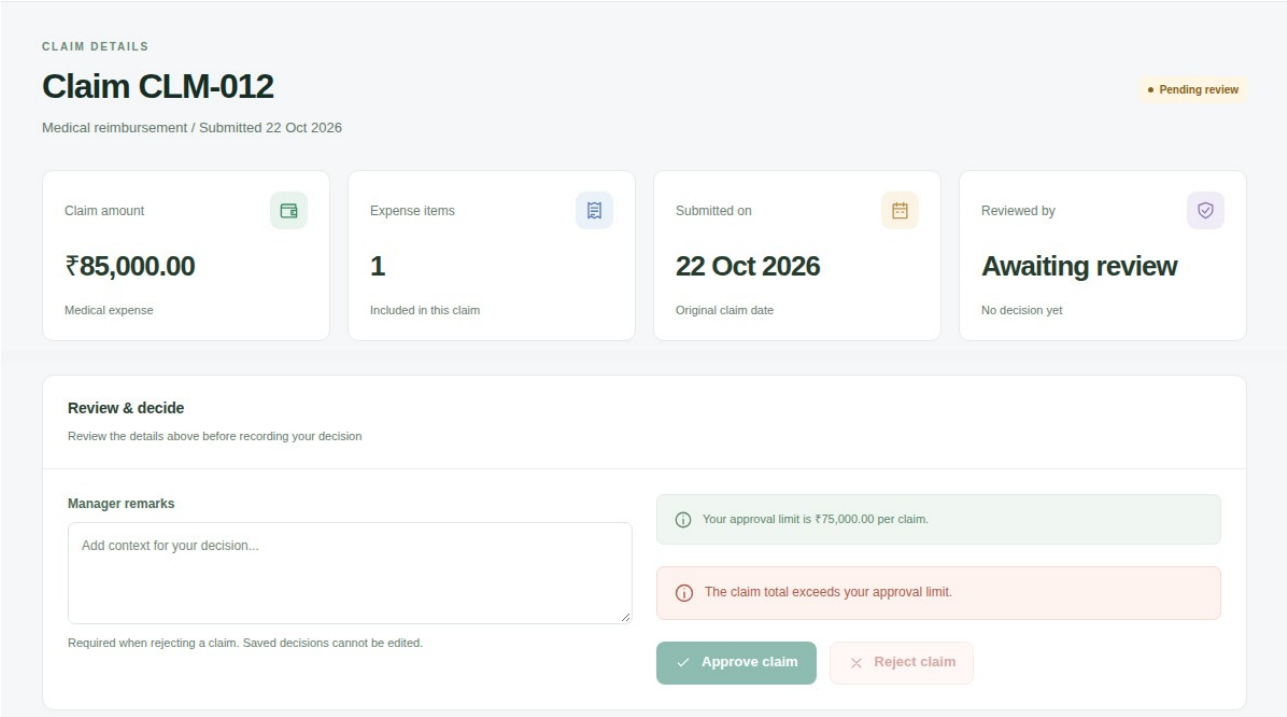


Figure 5. A claim above the manager's approval limit: the form shows why, and the Approve button is disabled.

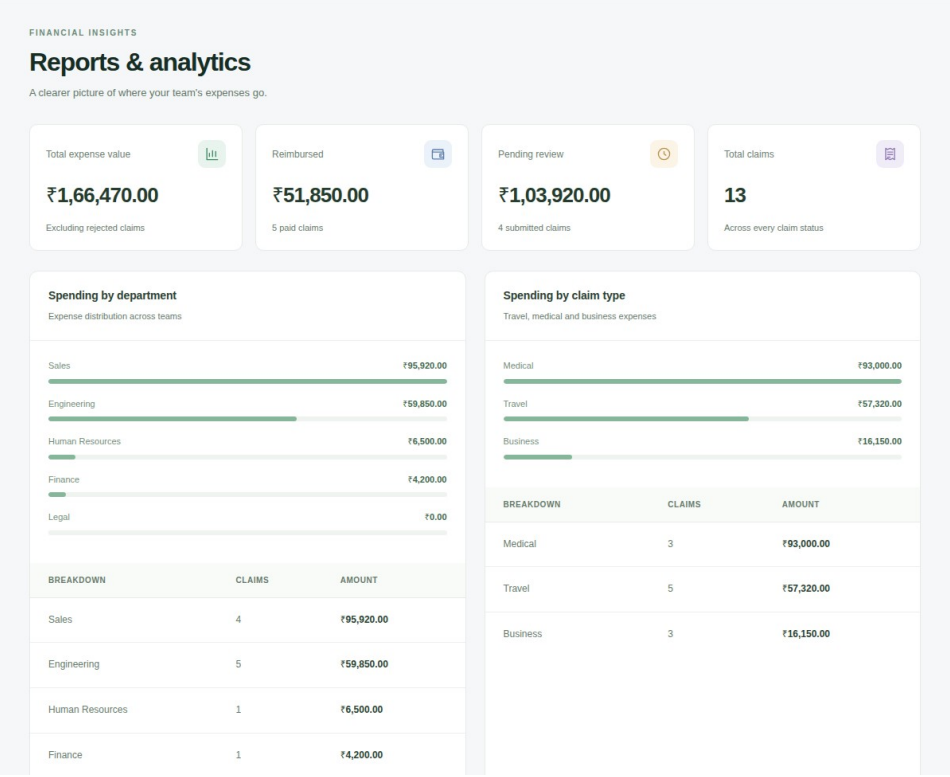


Figure 6. Reports for managers: totals by department and by claim type.

9 Conclusion

Starting from the ER diagram of the earlier assignment, we refined the design (seven changes), reduced it to ten relation schemas, listed the functional dependencies of each table and showed that every table is in BCNF, and therefore in 3NF. Normalizing a flat claim-form relation step by step produced the same tables as the ER reduction, which confirms the ER design. The design was implemented in MySQL with keys, foreign keys and CHECK constraints, and a working web application was built on top of it.